

CSC Problem Set 02 +X Report

Name: Nour Mahmoud

Email: nhmahmou@ncsu.edu

Section: 001

Collaborators:

Summary:

For the +X implementation, I enhanced the simulation by integrating a dynamic, real-time search tree visualization that clearly distinguishes expanded and frontier nodes, complete cost, heuristic, and score labels. I also added a control panel with interactive buttons to start, stop, reset, slow down, and speed up the simulation. I also added a map loader button that opens a file chooser to pick the next map. Additionally, I ensured reset logic to ensure the simulation resets completely with the robot returning to its start tile. Therefore, it makes the simulation more user friendly and easier to use and understand. Overall, these additions focused on enhancing the user's usability of the simulation through improving debugging efficiency and overall learnability of the user interface.

"+X" Concept:

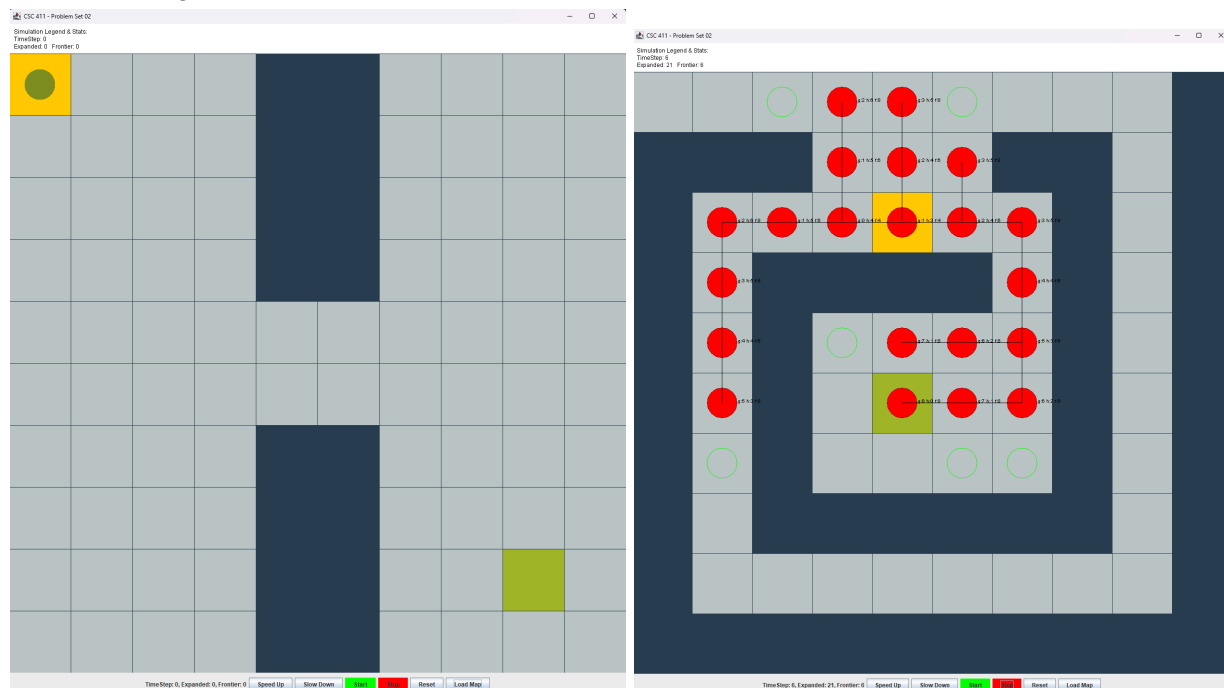
For my +X implementation, I enhanced the simulation's interactivity and debugging capabilities by adding a dynamic real-time search tree visualization along with a panel to control the simulation. The visualization overlays the grid with clearly distinguishable nodes: red filled circles represent expanded nodes while green circles represent frontier nodes. Each node displays its cost (g), heuristic (h), and total estimated cost (f) values, which provides valuable information regarding the A* search algorithm's decision making process as it navigates towards the target. Moreover, I implemented comprehensive controls including buttons for speeding up, slowing down, starting, stopping, resetting the simulation, and loading new maps. This allows users to interactively manage the simulation.

The design uses Java Swing for the GUI [3]. This allowed me to clearly display the buttons at the bottom of the screen and the statistics at the top of the frame. I drew inspiration from online visualizations like Red Blob's visualization of the A* algorithm [1]. These resources guided me in perfecting the UI and ensuring the simulation is beneficial to the user. Overall, the implementation creates an interaction environment that significantly enhances the learning and debugging experience, which makes it easier to analyze the A* algorithm I have implemented.

Technical Implementation of +X:

To implement the enhanced functionality, I restructured the simulation's GUI by creating a control panel that fully customizes the user's experience using Java Swing. The frame is divided into three sections: a header that displays real time statistics like the TimeStep, number of

expanded nodes, and number of frontier nodes ; a main grid area that visualizes the environment, robot, and overlay of the search tree; and labels at the bottom of the frame for the buttons. The search tree visualization differentiates between expanded nodes, shown in red, and frontier nodes, shown in green. Each node is annotated with its cost (g), heuristic (h), and total estimated cost (f).



I leveraged the structure of the A* algorithm to visually differentiate between the nodes that have been processed (expanded in red) and those that are pending evaluation (frontier in green). During the A* search, every node generated is encapsulated in its own node that stores its position, cost, heuristic, and overall priority. As nodes are evaluated, they are removed from the priority queue (frontier) and added to a static list of expanded nodes. Simultaneously, a snapshot of the current frontier is maintained in another static list. This allows the simulation to clearly indicate which parts of the search space have been fully explored and which parts are still under consideration. Finally, after the search is over, both lists in the Robot class are updated one last time.

```

List<Node> expandedNodes = new ArrayList<>();
List<Node> frontierNodes = new ArrayList<>();

HashMap<Position, Position> cameFrom = new HashMap<>();
HashMap<Position, Double> costSoFar = new HashMap<>();

cameFrom.put(start, null);
costSoFar.put(start, 0.0);

while (!frontier.isEmpty()) {
    Node current = frontier.poll();
    expandedNodes.add(current);

    frontierNodes = new ArrayList<>(frontier);

    if (current.position.equals(goal)) {
        List<Position> path = reconstructPath(cameFrom, start, goal);
        Robot.LastExpandedNodes = expandedNodes;
        Robot.LastFrontierNodes = frontierNodes;
        return path;
    }
}

```

To visualize these nodes on the tiles, I modified the paintComponent method. In this method, I iterate over the two lists and render each node using distinct colors and styles: expanded nodes are drawn as red filled circles, while frontier nodes are drawn as green outlined circles. Each node is positioned relative to its grid coordinate by computing its center based on the tile's size, and a label is drawn next to each expanded node displaying g, h, and f values. The use of two separate lists and distinct rendering styles not only helps in debugging and analyzing the A* algorithm's behavior in real-time but also provides educational insights into how the algorithm expands its search frontier and ultimately converges on the target [4]. Buttons used at the bottom of the frame are largely the same structure as the first problem set; therefore, they will not be further analyzed.

```

if (Robot.LastExpandedNodes != null) {
    //for every node in the list of expanded nodes, find the x and y position and calculate the center
    //by finding the tile size
    for (Robot.AStar.Node node : Robot.LastExpandedNodes) {

        //this finds the center of the circle (x and y positions) by converting the position index
        //to a coordinate and then centering the coordinate vertically and horizontally within the tile by
        //adding tileSize / 2
        int centerX = node.position.getCol() * TILESIZe + TILESIZe / 2;
        int centerY = node.position.getRow() * TILESIZe + TILESIZe / 2 + INTERNAL_HEADER_HEIGHT;
        int radius = TILESIZe / 4;

        //SETS THE RED CIRCLE
        int diameter = 2 * radius;
        g.setColor(Color.RED);
        g.fillOval(centerX - radius, centerY - radius, diameter, diameter);
        g.setColor(new Color(150, 0, 0));
        g.drawOval(centerX - radius, centerY - radius, diameter, diameter);

        //STATS
        double gVal = node.cost;
        double hVal = node.priority - node.cost;
        double fVal = node.priority;

        String label = String.format("g: %.0f h: %.0f f: %.0f", gVal, hVal, fVal);

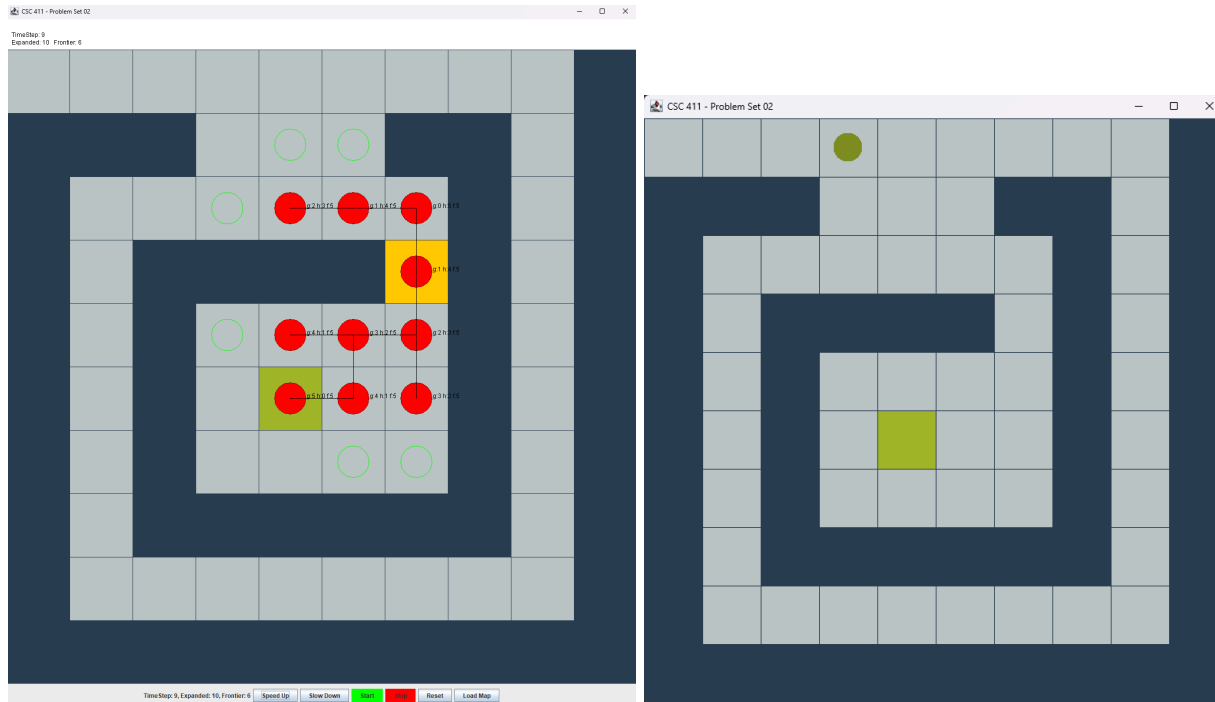
        g.setColor(Color.BLACK);
        g.drawString(label, centerX + radius + 2, centerY);

        //draw the connecting lines from parent to child if parent exists
        if (node.parent != null) {
            int parentCenterX = node.parent.position.getCol() * TILESIZe + TILESIZe / 2;
            int parentCenterY = node.parent.position.getRow() * TILESIZe + TILESIZe / 2 + INTERNAL_HEADER_HEIGHT;
            g.drawLine(centerX, centerY, parentCenterX, parentCenterY);
        }
    }
}

```

Evaluation and Results:

The addition of the +X enhancement produced a significant improvement in the simulation's usability and value. Prior to the enhancement, the simulation only displayed the robot moving on a grid with static tile colors, leaving the inner workings of the A* search algorithm hidden from the user. With the new implementation, the simulation now overlays a dynamic, real time visualization of the search tree. Expanded nodes are shown as red filled circles and are annotated with their cost (g), heuristic (h), and total (f) values, while frontier nodes appear as green outlined circles. This clear differentiation allows users to immediately understand which parts of the search space have been explored and which remain candidates for expansion, thereby facilitating easier debugging and understanding of the inner workings.



Feature	Without +X	With +X
Search Process Visibility	No representation	Real time visualization of expanded and frontier nodes
Debugging capabilities	Only final path output	Immediate insight into the search tree's state with buttons to help pause if needed
User controls	No control panel	Full control panel for speed up, slow down, start, stop, reset, and map loading

Moreover, the simulation layout now shows the control panel at the bottom with centered buttons, a header displaying real time statistics, and the main grid of tiles where the robot

moves. In summary, the +X enhancements have not only transformed the simulation into a more interactive and visually informative tool but have also greatly improved its educational and debugging value. By overlaying the search tree with clear distinctions between expanded and frontier nodes and integrating a comprehensive control panel, users can now observe and manipulate the inner workings of the A* algorithm as it operates. This interactive loop empowers users and helps better understand the algorithm's decision-making process, ultimately providing a robust platform for both learning and further experimentation in robotic search strategies.

Works Referenced:

1. Red Blob Games, "A* Pathfinding: Introduction," [Online]. Available: <https://www.redblobgames.com/pathfinding/a-star/introduction.html#astar>. Accessed: 01/28/2025.
2. Algo-VZ, "Algo-VZ," [Online]. Available: <https://algo-vz.netlify.app/>. Accessed: 01/28/2025.
3. Oracle, "Creating a GUI With JFC/Swing," Oracle Documentation, [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing/>. Accessed: 01/28/2025.
4. Stack Overflow, "How to draw a circle with given x and y coordinates as the middle spot of the circle," [Online]. Available: <https://stackoverflow.com/questions/19386951/how-to-draw-a-circle-with-given-x-and-y-coordinates-as-the-middle-spot-of-the-circle>. Accessed: 01/28/2025.